National University of Singapore
School of Computing
CS1101S: Programming Methodology
Semester I, 2026/2027

R12
Metacircular Evaluator

Getting started

This reflection modifies the metacircular evaluator of Lectures L11B and L12A (Textbook Sections 4.1.1–4.1.4). The evaluator below is the one from the lectures, with a function `parse_and_evaluate` that parses a program string and evaluates it in the global environment. Only `evaluate` and `apply` are shown here; the playground link runs the complete evaluator.

```
def evaluate(component, env):
    if is_literal(component):
        return literal_value(component)
    elif is_name(component):
        return lookup_symbol_value(symbol_of_name(component), env)
    elif is_application(component):
        return apply(evaluate(function_expression(component), env),
                     llist_of_values(arg_expressions(component), env))
    elif is_operator_combination(component):
        return evaluate(operator_combination_to_application(component),
                        env)
    elif is_conditional(component):
        return eval_conditional(component, env)
    elif is_lambda_expression(component):
        return make_function(lambda_parameter_symbols(component),
                             None, None,
                             make_return_statement(lambda_body(component)),
                             env)
    elif is_sequence(component):
        return eval_sequence(sequence_statements(component), env)
    elif is_return_statement(component):
        return eval_return_statement(component, env)
    elif is_function_definition(component):
        return eval_function_definition(component, env)
    elif (is_pass_statement(component) or
          is_global_statement(component) or
          is_nonlocal_statement(component)):
        return None
    elif is_assignment(component):
        return eval_assignment(component, env)
    else:
        return error('unknown syntax -- evaluate', component)
def apply(fun, arguments):
    if is_built_in_function(fun):
        return apply_built_in_function(fun, arguments)
    elif is_compound_function(fun):
        locals = function_locals(fun)
        globals = function_globals(fun)
        unassigned_markers = map(lambda symbol: '*unassigned*', locals)
        global_markers = map(lambda symbol: '*global*', globals)
        result = evaluate(function_body(fun),
                          extend_environment(
                              append(function_parameters(fun),
                                     append(locals, globals)),
                              append(arguments,
                                     append(unassigned_markers,
                                            global_markers)),
                              function_environment(fun)))
        if is_return_value(result):
            return return_value_content(result)
        else:
            return None
    else:
        return error('unknown function type -- apply', fun)
```

For example:

```
print(parse_and_evaluate("x = 4\nx + 38"))
```

Problems:

1. Compare Python in the Source Academy and our evaluator when they evaluate the following program:

   ```
   y = 1
   def f():
       x = y
       y = 42
       return x
   f()
   ```

   Python reports `UnboundLocalError: cannot access local variable 'y' where it is not associated with a value`, whereas our evaluator returns the string `'*unassigned*'`. Explain the behaviour of the evaluator, and modify it such that it behaves like Python in this situation. Which of the evaluator's functions decides that `y` is a local name of `f`, and when does it do so: when the definition of `f` is evaluated, or each time `f` is called?

2. Our evaluator uses the strings `'*unassigned*'` and `'*global*'` as markers in frames. What if the user program uses such a string? Experiment with the following programs, and explain what you observe. Then modify the evaluator such that no user program can be mistaken for a marker.

   ```
   def f(a):
       return a
   f("*global*")

   x = "*global*"
   x
   ```

   Does the program `f("*unassigned*")` cause trouble? Check this both in our evaluator and after you have changed it as in the previous question.

3. In which order does our evaluator evaluate the arguments of a function application? Find out using the following program:

   ```
   def first(a, b):
       return a
   first(print("left"), print("right"))
   ```

   The order depends on `llist_of_values`, which uses `map`: it is whatever order the given `map` applies its function in. Write a version of `llist_of_values` that evaluates the arguments from left to right regardless of how `map` is implemented, and a version that evaluates them from right to left.

4. Our evaluator does not support the function `parse`, and therefore we cannot write a meta-metacircular evaluator!

- Add the function `parse` such that the following evaluates to the syntax tree of an application of `parse` to the string `"1"`:

```
parse_and_evaluate('''parse('parse("1")')''')
```

- The program above has three levels of quotes. Explain the meaning of each of them, from the outside in.
- (Challenge, if you have a day to spare and like this sort of thing) Use your evaluator to evaluate an evaluator! The simplified evaluator below is written with only the features that our evaluator supports; it has a unary `print`. It evaluates numbers, names, operators, conditional expressions, assignments, function definitions with a single `return` statement, and applications. Find out which built-in functions our evaluator needs in order to evaluate it, and add them. Then use our evaluator to evaluate the simplified evaluator, applied to a small program. Report your findings to your reflection tutor in Week 13: what problems did you encounter? How did you solve them?

The simplified evaluator:

```
def ref(items, n):
    return head(items) if n == 0 else ref(tail(items), n - 1)

# environments are linked lists of frames; a frame is a pair of the
# linked list of its names and the linked list of their values

def lookup(name, env):
    return (error(name, "unbound name") if is_none(env)
            else lookup_frame(name, head(env), env))

def lookup_frame(name, frame, env):
    return scan(name, head(frame), tail(frame), env)

def scan(name, names, values, env):
    return (lookup(name, tail(env)) if is_none(names)
            else head(values) if name == head(names)
            else scan(name, tail(names), tail(values), env))

def extend(names, values, env):
    return pair(pair(names, values), env)

def define(name, value, env):
    frame = head(env)
    set_head(frame, pair(name, head(frame)))
    set_tail(frame, pair(value, tail(frame)))
    return value

def symbols(nodes):
    return (None if is_none(nodes)
            else pair(ref(head(nodes), 1),
                      symbols(tail(nodes))))

def eval_all(exprs, env):
    return (None if is_none(exprs)
            else pair(evaluate(head(exprs), env),
                      eval_all(tail(exprs), env)))

def eval_sequence(stmts, env):
    first = evaluate(head(stmts), env)
    return (first if is_none(tail(stmts))
            else eval_sequence(tail(stmts), env))

def apply(fun, args):
    return (apply_in_underlying_python(ref(fun, 1), args)
            if head(fun) == "primitive"
            else evaluate(ref(fun, 2),
                          extend(ref(fun, 1), args, ref(fun, 3))))

def eval_operator(comp, operands, env):
    return apply(lookup(ref(comp, 1), env),
                 eval_all(operands, env))

def eval_conditional(comp, env):
    return (evaluate(ref(comp, 2), env)
            if evaluate(ref(comp, 1), env)
            else evaluate(ref(comp, 3), env))

def eval_definition(comp, env):
    name = ref(ref(comp, 1), 1)
    body = ref(ref(comp, 3), 1)
    fun = llist("function", symbols(ref(comp, 2)), body, env)
    return define(name, fun, env)
```

Version 1.0 — Built: 2026-11-04 18:06:56 SGT